Project 6 Elevator Finite State Machine Logic

Introduction

This document describes the design criteria for developing the Elevator's Finite State Machine (FSM) logic. This document will describe in detail the conditions for state transitions, exception behaviour, opposing condition handling, high level data-structures, and alternative finite logic depending on the given hardware.

The given hardware configuration of the elevator as of today (May 26, 2026) remains not fully understood, and as such this document describes the implementation of logic for a couple varying hardware configuration if the probable default proves impossible to use.

Equipment Description

The elevator project comprises several subsystems that the FSM must manage to ensure proper coordination across all equipment. Major systems include the supervisor controller, motor controller, floor controllers, and car controller.

From a visual inspection of the Elevator 1 equipment, a couple of notable observations were made:

1. There does not appear to be a level sensor per floor, instead there appears to be a single photo eye located at the top of each elevator shaft.
2. There does not appear to be directional floor commands.

This implies two important design decisions that appear to conflict with some of the stated challenge features as such while their logic is laid out in this document specific implementation details will not be discussed until further development confirms the specific architecture used.

First, the lack of a level sensor for each floor implies that the car height is controlled via an encoder. Second, the lack of a floor directional command simplifies the FSM logic: where each floor button calls the car to the floor and then the destination is determined by the car. However, using optional directional logic will be specified.

FSM Logical Operations

This section describes the order, priority, abstract data structures, and exception handling without specifically detailing the full FSM. It's designed to allow programmers to quickly

understand the order at which requests are served, how requests are received and stored, and how the FSM should respond to a given input to maintain a defined state at all times.

FSM Design Decisions

Standard Elevator

The standard elevator assumes the following constraints:

- The floor call button calls the car to a floor (NOT DIRECTIONAL)
- Motor Control Unit uses encoder to determine elevator height

Following these constraints, the following FSM produces reliable deterministic behaviour.

General Controller Area of Responsibilities

Supervisory Controller

- Network communication
- Front-End hosting
- Floor and car scheduling – first come first serve basis
- Calculate and provide delta distance between floors and dispatch the car to floor

Floor Controllers

- Send car request to supervisory controller for a given floor
- If individual sensors are provided on a per-floor basis floor specific sensor devices should be handled by the designated floor controller

Car Controller

- Send selected destination floor
- Send current floor to calculate differential position require to move by the motor controller

Motor Controller

- Receives car commands from supervisory controller, dispatches the car the calculated delta distance
- Takes in emergency stop logic for photo-eye limit sensors

Supervisory Controller States and Conditionals

The supervisory controller (SC) acts as the main coordination and managing hub. It collects and orchestrates data collected via the subsystems and controls how the main system works. Centralizing the code controlling the system inside the Raspberry Pi (RPi) has notable advantages. Since the Pi also handles front end and database management, all command and control can be run on a single device with minimal downtime caused by sending and receiving messages.

While this document does not describe integration of the database applications in this project, future revisions could be added as required.

The SC by default will follow requests in the following order:

1. Physical button requests from the local lan
2. Network button requests via a remote connection

This follows the principle that in-person requests have a higher priority than remote commands. Floor requests sent (whether physical or via remote connections) are handled using a queue system. Where the first request is handled completely before the next request can be executed.

From a high level the elevator should work in the following machine state:

Current State	Condition	Result State	Action
Idle	Floor request is received or pulled from queue	Collection	A floor request has been received but a car command has not been made.
Collection	1. Car command is received 2. No car command is received within 10s of arrival at destination	1: Delivery 2: Idle	1: The car is still considered to be in use by users, new floor requests are added to queue but are not executed until after the delivery state is complete. 2: The car is assumed to have been mis-called/the user opted for the stairs and will pull the next job from the queue. Returns to idle and repeat FSM.
Delivery	Car reaches car command floor	Idle	The car reaches the user's requested floor, and the cargo is considered "delivered". Upon completion the system returns to the idle state.

State Descriptions

IDLE

The idle state is the default state of the elevator, during this state the elevator will actively look for new jobs to execute. Jobs will be pulled from two queues, one for dealing with physical requests from the CAN network, and the other for dealing with remote requests from the RDP. Priority given to the physical request queue over remote requests. This is done with the understanding that people physically at the elevator should take priority over a convenience feature such as remote floor requests

COLLECTION

The collection state defines the state wherein the elevator has taken a job from the queue (whether physical or remote) and is moving to the requested floor to pick up the requesting user.

If the elevator arrives at the requested floor and does not receive a car command within 10 seconds (a user requested destination floor from the car controller), the elevator returns to the idle state and will look to grab the next job from the available queues.

DELIVERY

Delivery is the state of the elevator after the requester enters the elevator provides a car command. The elevator will move to the requested floor, after which the elevator returns to the idle state.